Universidade Federal de Santa Maria
Departamento de Computação Aplicada
Prof. Cesar Tadeu Pozzer
Disciplina: Computação Gráfica Avançada
pozzer@inf.ufsm.br
28/10/2025

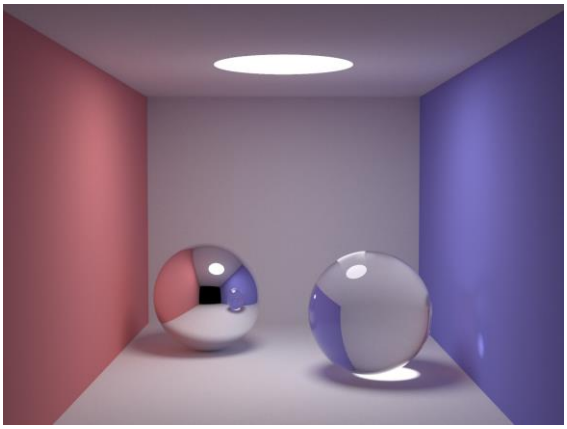
Path Tracing (por Gabriel Di Domenico, Cesar Pozzer)

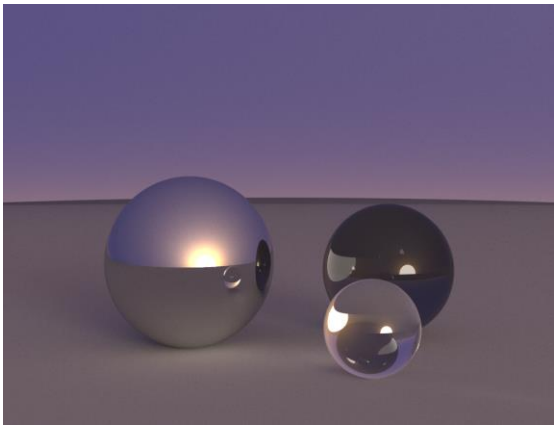
Path tracing - PT (The rendering Equation, ACM, James Kajiya) é uma técnica que consiste na simulação do traçado de todos os raios de luz presentes em uma cena, calculando a cor em cada um dos pontos que a luz toca em cada objeto, trazendo reflexões realistas, sombras corretamente posicionadas e com intensidades reais e cores que correspondem à iluminação do ambiente. Infelizmente na atualidade ainda não existe poder computacional para conseguir realizar esse cálculo, então algoritmos de PT utilizam de métodos estatísticos, como o método Monte Carlo, para realizar um cálculo próximo do real para cada pixel de cor em uma cena.

Designa-se por método de **Monte Carlo (MMC)** qualquer método de uma classe de métodos estatísticos que se baseiam em amostragens aleatórias massivas para obter resultados numéricos. Em suma, utilizam a aleatoriedade de dados para gerar um resultado para problemas que a priori são determinísticos. O método de Monte Carlo tem sido utilizado há muito tempo como forma de obter aproximações numéricas de funções complexas em que não é viável, ou é mesmo impossível, obter uma solução analítica ou, pelo menos, determinística. [Wikipedia]



Aplicação do método de Monte Carlo para determinar a área de um lago.





Por mais complexa que possa ser a implementação de PT, na atualidade é possível encontrar sua utilização em jogos como o cyberpunk 2077, como visto neste vídeo, porém para ter resultados de desempenho

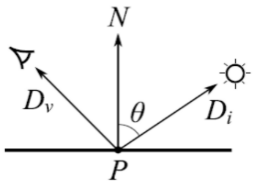
aceitáveis para um jogo de FPS é necessário o melhor hardware do mercado atual, RTX 4090 ou RTX 5090, utilizando resoluções com QuadHD ou FullHD, 4K ainda é um futuro distante.

Neste material foi utilizado como base o post sobre PT da NVIDIA, neste link (faz comparações entre rasterização, ray tracing e path tracing), o material de PT em 99 linhas de código proposto por Kevin Beason, nomeado como smallpt, e o material tutorial proposto por David Cline, onde pode ser acessado neste link. (arquivo cga_14_SmallPT.ppt)

Path tracing segue os três princípios de óptica:

- Princípio da iluminação global – Dada uma cena, todo o objeto presente no local deve contribuir com a iluminação de qualquer outro objeto.
- Princípio de equivalência – Não existe nenhuma distinção a ser feita entre iluminação emitida por uma fonte de luz e uma iluminação refletida por uma superfície.
- Princípio da direção – A iluminação originada de uma superfície deve espalhar em alguma direção em função da direção de entrada da iluminação que chega e da direção de saída que está sendo amostrada. (BRDF)

Para conseguir alcançar tais objetivos, **Kajiya** propôs a Rendering Equation (já apresentada no início do texto). Essa equação da forma que foi construída originalmente seria impossível de computar com hardware que temos na atualidade, uma vez que é necessário calcular uma integral que leva em consideração todos os raios possíveis que contribuem com a cor do local atingido. Como dito anteriormente, para transformar essa equação em algo possível de ser calculado com computadores atuais é necessário o uso de métodos de probabilidade para alcançar resultados aproximados do ideal. O método de Monte Carlo é o mais utilizado, pois sua abordagem baseada em probabilidades permite estimar o valor de funções complexas por meio de múltiplas amostragens aleatórias. Para garantir que o cálculo seja estatisticamente correto, cada amostra deve ser ponderada pela sua probabilidade de seleção.



Atenuação Angular, representa o peso da luz que chega no ponto de intersecção. A relação do cosseno do ângulo entre o raio incidente e a normal influência na intensidade da luz refletida

Quantidade de luz emitida em direção a ponto Y

Distribuição de Fresnel, modela a quantidade de luz refletida em uma superfície, em relação a ângulo de incidência D_i

$$\widehat{L}(P \to D_v) = L_e(P \to D_v) + \frac{F_s(D_v, D_i)|\cos\theta|\widehat{L}(Y_i \to -D_i)}{p^{tot}_{angle}(D_i)}$$

Alterações para utilizar o método Monte Carlo para calcular o valor aproximado disparando um novo raio em uma direção aleatória e realizando o cálculo da equação novamente

Quantidade de luz total do ponto P em direção a D_v

Quantidade de luz emitida no ponto P em direção D_v

Esse termo aparece porque a equação está sendo resolvida com método de Monte Carlo. Como o Path Tracing usa amostragem aleatória, ele precisa dividir pela probabilidade de escolher um determinado raio D_i Isso garante que os raios mais prováveis não dominem os resultados de forma errada.

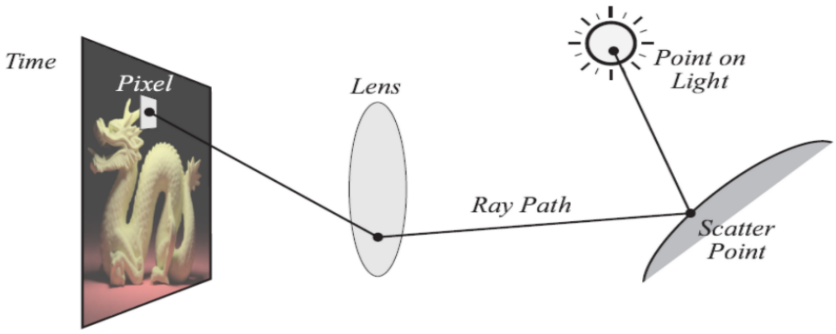
Um algoritmo de Path Tracing pode ser customizado alterando a quantidade de samples no método de Monte Carlo, melhorando a imagem com a diminuição da granulação dela, porém, aumentando o tempo de execução. Isso ocorre pois quanto maior o número de sample, maior o número de raios traçados, e em uma cena quanto mais raios traçados maior a probabilidade de um acertar a fonte observadora, efetivando a cor dele.

O link https://www.kevinbeason.com/smallpt/#moreinfo contém slides com informações detalhadas do funcionamento do algoritmo, feitos por David Cline, da Oklahoma State University.

```
for each pixel (i,j) do
    Vec3 C = 0
    for (k=0; k < samplesPerPixel; k++) do
        Create random ray in pixel:
            Choose random point on lens P_lens
            Choose random point on image plane P_image
            D = normalize(P_image − P_lens)
            Ray ray = Ray(P_lens, D)
        castRay(ray, isect)
        if the ray hits something then
            C += radiance(ray, isect, 0)
        else
            C += backgroundColor(D)
        end if
    end for
    image(i,j) = C / samplesPerPixel
end for
```

Função não recursiva

Função recursiva

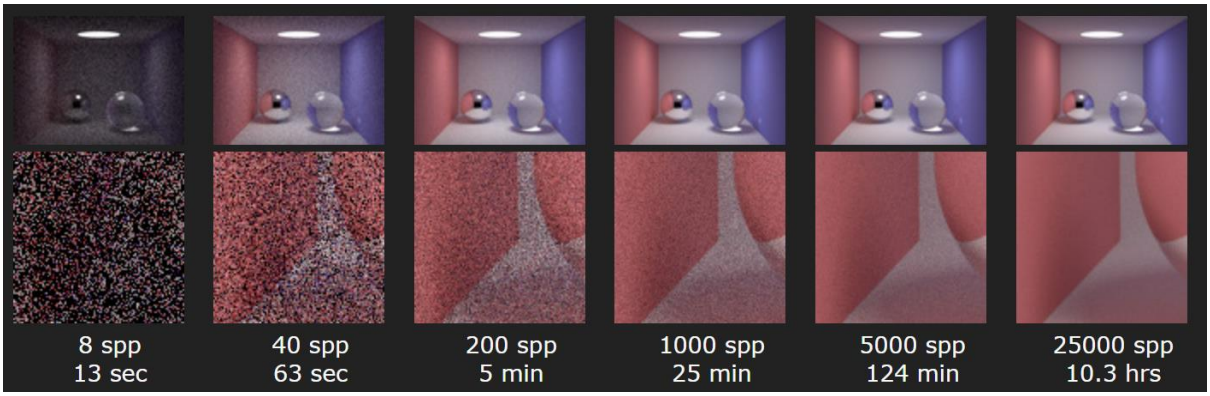


Quando os raios são traçados a partir da câmera, método é chamado de Backwards path tracing. Para gerar alguma cor é necessário que em N samples pelo menos um raio consiga atingir uma fonte de luz, caso contrário o pixel da imagem ficará preto (ou próximo do preto).

Algoritmos de PT podem traçar seus raios a partir de uma ou mais fontes de luz. Neste caso o método é chamado de Forward path tracing ou Light tracing. Nesta abordagem calcula-se a colisão dos raios dentro da cena até que em algum momento ele alcance a câmera. Para se aprofundar mais na dispersão dos raios sugiro o vídeo explicativo publicado pelo canal Josh's Channel, disponível o link aqui.

O SmallPT (https://www.kevinbeason.com/smallpt/) é um algoritmo acessível e compacto que consegue gerar uma imagem utilizando o método de PT para alcançar níveis de detalhe realísticos. O código possui somente 99 linhas e é open source para qualquer um utilizar ou incrementar funcionalidades. Antes da sua execução é possível selecionar a quantidade de samples, podendo-se assim ajustar a granularidade da imagem e o tempo de execução.

Os resultados e tempos na imagem com diferentes amostras por pixel (samples per pixel - spp) foram obtidos com um computador utilizando um Intel Core 2 Quad 2.4GHz de 4 núcleos. SmallPT é um algoritmo que roda somente em CPU, dentro da página inicial é possível acessar links que levam para implementações das mais diversas implementações.



Podemos destacar as classes do SmallPT desta forma:

- Ray: Calcula a propagação do raio
- Vec: Classe que representa os vetores, usada para calcular ponto, normal e cores
- Refl_t: Estrutura que guarda o tipo de reflexão
- Sphere: Classe que representa a esfera da imagem, o código SmallPT consegue somente traçar os raios em esferas.
- spheres: Estrutura que representa a cena
- intersect: Função responsável de calcular a intersecção entre os objetos na cena
- radiance: Função recursiva que calcula a radiância da rendering equation
- main: Função que inicia o programa e o loop principal que percorre cada pixel da imagem.

Nos demos da disciplina são apresentadas versões do SmallPT para Code::Blocks e Visual Studio. Em Code::Blocks faz uso da função erand48 (https://www.ibm.com/docs/en/zos/2.5.0?topic=functions-erand48-pseudo-random-number-generator) para geração de números pseudo-aleatórios (não disponível no compilador MinGW para Windows), e também de paralelismo com OpenMP. O resultado é muito semelhante entre as duas versões.

Em https://github.com/matt77hias/smallpt estão disponíveis versões do código para diferentes linguagens, incluindo C, C++, C#, Python, Haskell, GLSL, Java dentre muitas outras.

Single process - Single threaded

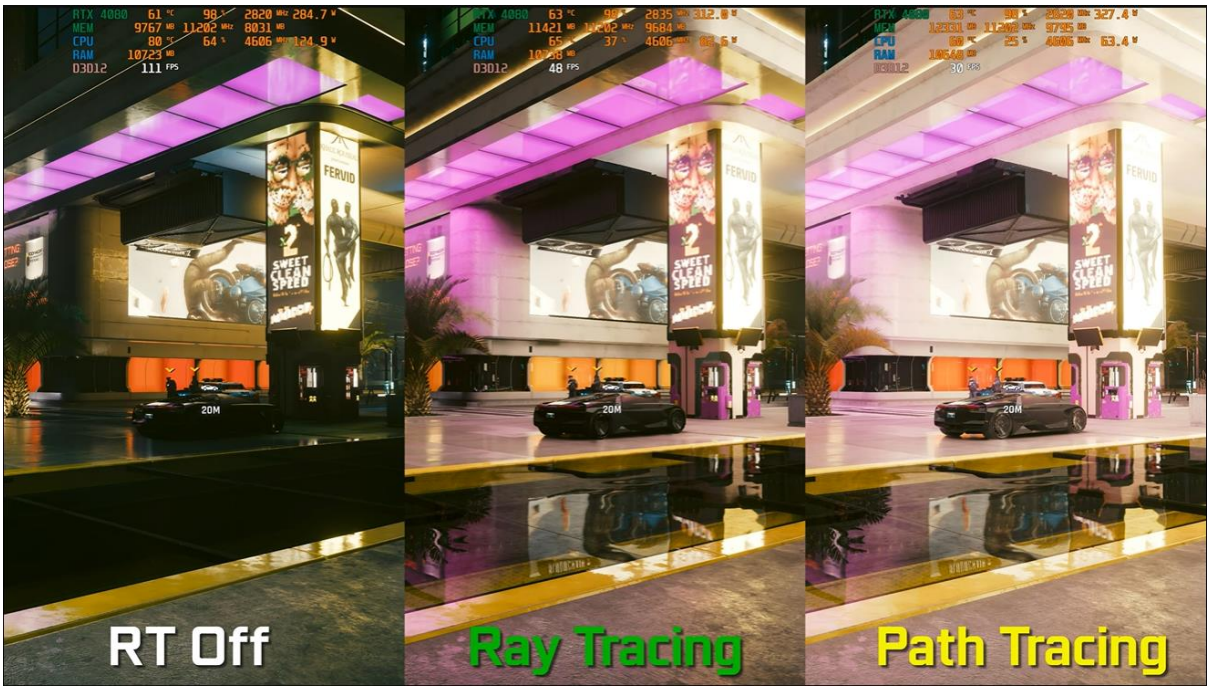
Programming Language	Compiler/Interpreter	Wall clock (64 spp)
C	MSVC++ 15.5.9	00h 01m 51.88s
C++	MSVC++ 15.5.9	00h 01m 21.32s
C#	CLR 4.6.1055.0	00h 01m 30.56s
CoffeeScript	CC 1.11.1/Chrome	00h 04m 19.63s
Python 2.7	PyPy 5.6.0	00h 20m 47.33s

Multi process - Multi threaded

Programming Language	Compiler/Interpreter	Wall clock (64 spp)
C + OpenMP	MSVC++ 15.5.9	00h 00m 22.09s
C++	MSVC++ 15.5.9	00h 00m 16.71s
C++ + OpenMP	MSVC++ 15.5.9	00h 00m 25.97s
C#	CLR 4.6.1055.0	00h 00m 24.16s
CUDA	MSVC++ 15.5.9/NVCC 9.2	00h 00m 07.89s

Exemplo de jogos que fazem uso de Path Tracing: Cyberpunk 2077, Indiana Jones and the Great Circle, Black Myth: Wukong, e Portal 2. Seguem exemplos comparativos com e sem path tracing.

Cyberpunk 2077



Indiana Jones and the Great Circle



Black Myth: Wukong



Portal 2

